IA Pessoal Tipo Jarvis — Planejamento Completo

1. Visão Geral do Projeto

A proposta é criar uma **IA pessoal local**, semelhante a um “Jarvis”, capaz de:

- conversar com o usuário por chat
- responder perguntas
- lembrar informações importantes
- executar comandos e automações no futuro
- consultar arquivos e dados locais
- funcionar localmente, com mais privacidade
- evoluir para uma arquitetura mais robusta com banco de dados e múltiplos módulos

---

2. Stack Principal

Linguagem Principal

Python

Motivo:

- simples de desenvolver
- enorme ecossistema para IA
- ótima integração com APIs, banco de dados e automações
- excelente suporte para LLMs locais e frameworks web

Backend / API

FastAPI

Motivo:

- rápido
- moderno

- fácil de organizar
- ótimo para criar endpoints de chat, memória, arquivos, comandos e integrações

Interface Web

Gradio ou **Streamlit**

Gradio

Ideal se você quer:

- criar interface de chat muito rápido
- testar a IA rapidamente
- montar protótipos com poucas linhas

Streamlit

Ideal se você quer:

- uma interface mais parecida com dashboard
- múltiplas páginas
- exibir memória, logs, configurações, status do sistema

Modelo Local

Ollama ou **llama.cpp**

Ollama

Ideal para:

- começar mais rápido
- rodar modelos locais com facilidade
- testar modelos como Llama, Mistral, Gemma, Phi, etc.

llama.cpp

Ideal para:

- maior controle técnico
- ambientes mais otimizados

- uso com GGUF e ajustes finos de desempenho

Banco de Dados Inicial

SQLite

Motivo:

- leve
- simples
- sem configuração complexa
- perfeito para MVP

Banco Futuro

PostgreSQL

Motivo:

- melhor para escala
- múltiplos usuários
- mais performance
- recursos avançados de consultas e integrações

---

3. Objetivo da Arquitetura

A IA deve ser separada em módulos, para ficar fácil de evoluir.

Estrutura conceitual:

1. **Interface**
   Onde o usuário conversa com a IA
2. **API / Backend**
   Onde a lógica principal acontece
3. **Módulo do modelo local**
   Onde a IA gera respostas
4. **Memória / Banco de dados**
   Onde ficam histórico, preferências e dados importantes

5. **Módulo de ferramentas**
   Onde ficam comandos, leitura de arquivos, automações, pesquisas locais etc.
6. **Camada de contexto**
   Responsável por montar o prompt com histórico, memória e instruções

---

4. Arquitetura Recomendada

Estrutura em alto nível

```
Usuário
  ↓
Interface Web (Gradio ou Streamlit)
  ↓
FastAPI
  ↓
Serviço Orquestrador da IA
  ├── Gerenciador de Prompt
  ├── Gerenciador de Memória
  ├── Serviço do Modelo Local (Ollama/llama.cpp)
  ├── Ferramentas / Ações
  └── Banco de Dados (SQLite)
```

---

5. Estrutura de Pastas Recomendada

```
jarvis_ai/
|
├── app/
|   ├── main.py
|   ├── api/
|   |   ├── routes_chat.py
|   |   ├── routes_memory.py
|   |   ├── routes_tools.py
|   |   └── routes_health.py
|   |
```

```
│   ├── core/
│   │   ├── config.py
│   │   ├── database.py
│   │   ├── security.py
│   │   └── constants.py
│   │
│   ├── models/
│   │   ├── chat.py
│   │   ├── memory.py
│   │   └── user.py
│   │
│   ├── schemas/
│   │   ├── chat_schema.py
│   │   ├── memory_schema.py
│   │   └── tool_schema.py
│   │
│   ├── services/
│   │   ├── llm_service.py
│   │   ├── memory_service.py
│   │   ├── prompt_service.py
│   │   ├── tool_service.py
│   │   └── assistant_service.py
│   │
│   ├── repositories/
│   │   ├── chat_repository.py
│   │   └── memory_repository.py
│   │
│   ├── tools/
│   │   ├── file_reader.py
│   │   ├── notes_tool.py
│   │   ├── system_tool.py
│   │   └── web_tool.py
│   │
│   └── utils/
│       ├── logger.py
│       ├── helpers.py
│       └── date_utils.py
│
```

```
├── interface/
│   ├── gradio_app.py
│   └── streamlit_app.py
│
├── data/
│   ├── app.db
│   ├── uploads/
│   └── logs/
│
├── tests/
│   ├── test_chat.py
│   ├── test_memory.py
│   └── test_tools.py
│
├── requirements.txt
├── .env
├── .gitignore
└── README.md
```

---

6. Passo a Passo Completo para Criar a IA

ETAPA 1 — Definir o escopo do MVP

Antes de programar, defina o que sua IA fará na primeira versão.

MVP recomendado

Na primeira versão, sua IA deve conseguir:

- conversar em chat
- responder usando modelo local
- guardar histórico de conversa
- salvar memórias simples
- ler instruções do sistema
- ter interface web básica
- funcionar localmente

Não colocar ainda no MVP

Para não complicar no começo, deixe para depois:

- reconhecimento de voz
- geração de voz
- automações avançadas do PC
- controle de desktop
- acesso completo à internet
- múltiplos usuários
- sistema de permissões complexo

Resultado esperado da etapa

Ao final, você terá uma visão clara:

- o que vai entrar agora
- o que vai ficar para versões futuras

ETAPA 2 — Preparar o ambiente de desenvolvimento

Instalar Python

Use Python 3.11 ou superior.

Verifique:

```
python--version
```

Criar ambiente virtual

```
python-m venv venv
```

Ativar ambiente virtual

Windows

```
venv\Scripts\activate
```

Linux / Mac

```
source venv/bin/activate
```

Instalar dependências iniciais

```
pip install fastapi uvicorn sqlalchemy pydantic python-dote
nv requests
```

Se for usar Gradio:

```
pip install gradio
```

Se for usar Streamlit:

```
pip install streamlit
```

Criar requirements.txt

```
pip freeze > requirements.txt
```

ETAPA 3 — Escolher como rodar o modelo local

Você pode escolher entre **Ollama** e **llama.cpp**.

ETAPA 4 — Configurar Ollama

Quando escolher Ollama

Escolha Ollama se você quer facilidade.

Instalação

Instale o Ollama no sistema operacional.

Depois teste:

```
ollama--version
```

Baixar um modelo

Exemplo:

```
ollama pull llama3
```

Outros modelos possíveis:

- mistral
- phi
- gemma
- llama3
- codellama

Testar no terminal

```
ollama run llama3
```

Objetivo dessa etapa

Garantir que o modelo local responde no seu computador antes de integrar no FastAPI.

ETAPA 5 — Configurar llama.cpp

Quando escolher llama.cpp

Escolha se quiser mais controle e otimização.

Fluxo básico

- baixar binário ou compilar
- baixar modelo em formato GGUF
- rodar localmente
- integrar com Python

Observação

Para começar, **Ollama costuma ser mais simples**.

Recomendação prática

Comece com **Ollama** e futuramente teste **llama.cpp** se precisar de mais desempenho ou customização.

ETAPA 6 — Criar o backend com FastAPI

Objetivo

Criar a base da API que receberá mensagens e devolverá respostas.

Arquivo principal

`app/main.py`

O que esse arquivo deve fazer

- iniciar o FastAPI
- registrar rotas
- configurar CORS se necessário
- disponibilizar endpoints de health check

Exemplo de responsabilidades

- `/health` → verificar se a API está funcionando
- `/chat` → enviar pergunta e receber resposta
- `/memory` → salvar e consultar memórias
- `/tools` → usar ferramentas futuramente

ETAPA 7 — Criar endpoint de saúde da aplicação

Objetivo

Confirmar se a API está viva.

Exemplo de rota

```
GET /health
```

Resposta esperada

```
{
  "status":"ok"
}
```

Importância

Esse endpoint ajuda muito no começo, porque permite testar rapidamente se o servidor está rodando certo.

ETAPA 8 — Criar o serviço de integração com o modelo

Objetivo

Separar a lógica de chamada ao modelo local.

Crie um serviço:

`app/services/llm_service.py`

Responsabilidades desse serviço

- enviar prompt ao Ollama ou llama.cpp
- receber resposta
- tratar erro de conexão
- permitir trocar de modelo facilmente no futuro

Ideia principal

A API não deve conversar diretamente com o modelo em todos os lugares.

Ela deve usar um serviço central.

Exemplo do fluxo

1. usuário manda mensagem
2. endpoint `/chat` recebe
3. `assistant_service.py` organiza contexto
4. `llm_service.py` chama o modelo

5. resposta volta para o usuário

ETAPA 9 — Criar o sistema de prompt

Objetivo

Definir a personalidade da sua IA.

Crie um serviço:

`app/services/prompt_service.py`

Esse serviço deve montar:

- instrução principal da IA
- contexto do usuário
- histórico recente
- memórias importantes
- mensagem atual

Exemplo de conteúdo do prompt base

- nome da IA
- comportamento educado
- linguagem clara
- foco em produtividade
- uso de memória quando necessário
- não inventar dados quando não souber

Benefício

Isso deixa sua IA mais consistente, parecendo realmente um assistente pessoal.

ETAPA 10 — Criar sistema de memória inicial

Objetivo

Guardar informações úteis do usuário.

Crie:

- tabela de memórias
- tabela de conversas
- serviço de memória

O que salvar no começo

- histórico de mensagens
- preferências do usuário
- dados importantes marcados
- data e hora das interações

Exemplo de memória

- nome do usuário
- preferências
- tarefas frequentes
- assuntos recorrentes

Importante

Nem tudo precisa virar memória.

No início, salve apenas itens relevantes.

ETAPA 21 — Implementar memória útil de verdade

Depois que o básico funcionar, melhore a memória.

Você pode separar memória em:

- memória curta: histórico recente da conversa
- memória longa: informações persistentes
- memória contextual: informações relevantes para a conversa atual

Estratégia simples

Buscar no banco apenas:

- últimas mensagens
- memórias com maior relevância
- memórias da mesma categoria do tema atual

Exemplo

Se o usuário falar de estudos, buscar:

- preferências de estudo
- matérias favoritas
- tarefas salvas relacionadas

ETAPA 22 — Criar comandos internos

Exemplo de comandos úteis

- “salve isso como memória”
- “mostre minhas memórias”
- “resuma esta conversa”
- “crie uma nota”
- “liste minhas tarefas”

Benefício

Sua IA começa a parecer realmente um assistente pessoal.

ETAPA 23 — Adicionar sistema de tarefas

Depois do MVP

Crie uma tabela de tarefas:

tabela `tasks`

- id
- title
- description

- status
- priority
- due_date
- created_at

Funções futuras

- criar tarefa
- concluir tarefa
- listar tarefas
- lembrar tarefas pendentes

ETAPA 24 — Adicionar ingestão de documentos

Objetivo

Permitir que a IA leia arquivos do usuário.

Primeira versão

- upload de `.txt`
- upload de `.md`
- upload de `.pdf` depois

Fluxo

1. usuário envia arquivo
2. sistema extrai texto
3. texto é salvo
4. IA usa esse conteúdo como contexto

Evolução futura

Depois você pode usar:

- chunking
- embeddings

- busca semântica
- RAG

ETAPA 25 — Preparar para RAG no futuro

O que é RAG

A IA consulta informações externas antes de responder.

No futuro você pode adicionar

- embeddings
- banco vetorial
- busca por similaridade
- recuperação de trechos relevantes de documentos

Ferramentas futuras possíveis

- ChromaDB
- FAISS
- PostgreSQL com pgvector

Importante

Não precisa colocar isso no dia 1.

ETAPA 26 — Melhorar a personalidade da IA

Para parecer um Jarvis, a IA precisa de consistência.

Você pode definir

- nome
- tom de fala
- estilo
- forma de responder
- nível de objetividade

- regras de comportamento

Exemplo de personalidade

- educado
- direto
- inteligente
- organizado
- prestativo
- focado em produtividade

Cuidado

Não tente exagerar demais no personagem no início.

Primeiro faça funcionar bem.

ETAPA 27 — Preparar migração de SQLite para PostgreSQL

Quando seu projeto crescer, faça a migração.

Sinais de que chegou a hora

- muitos dados
- múltiplos usuários
- buscas mais complexas
- necessidade de maior desempenho
- futuras integrações mais robustas

Como facilitar a migração

Desde o início:

- use SQLAlchemy
- mantenha repositórios separados
- não escreva SQL espalhado pelo projeto
- use migrations com Alembic no futuro

ETAPA 28 — Segurança básica

Mesmo local, pense em segurança.

Faça pelo menos isso

- validar entrada do usuário
- limitar tamanho de mensagem
- sanitizar conteúdo de arquivos
- controlar ferramentas perigosas
- proteger rotas sensíveis se houver autenticação

No futuro

- login
- sessões
- usuários
- permissões
- criptografia de dados sensíveis

ETAPA 29 — Testes

Testes mínimos

- endpoint `/health`
- endpoint `/chat`
- serviço de memória
- integração com banco
- falha do modelo local
- resposta com histórico

Benefício

Evita quebrar tudo a cada melhoria.

ETAPA 30 — Roadmap de evolução

Fase 1 — MVP local

- chat funcional
- modelo local
- FastAPI
- interface simples
- SQLite
- histórico
- memória básica

Fase 2 — Assistente útil

- tarefas
- notas
- leitura de arquivos
- comandos internos
- perfis de memória

Fase 3 — Assistente avançado

- RAG
- busca semântica
- voz
- automações
- integrações com calendário, e-mail e navegador

Fase 4 — Plataforma robusta

- PostgreSQL
- autenticação
- multiusuário
- painel admin

- plugins/ferramentas expansíveis

---

7. Melhor Escolha para Começar

Stack recomendada para sua primeira versão

Melhor combinação para MVP

- **Python**
- **FastAPI**
- **Streamlit** ou **Gradio**
- **Ollama**
- **SQLite**

Minha recomendação prática

Se quiser começar da forma mais simples e eficiente:

- **Backend:** FastAPI
- **Modelo local:** Ollama
- **Interface:** Streamlit
- **Banco:** SQLite

Motivo

Essa combinação entrega:

- simplicidade
- organização
- boa escalabilidade
- facilidade para evoluir depois

---

8. Ordem Ideal de Implementação

Siga esta ordem:

Semana / Etapa 1

- criar estrutura do projeto
- configurar ambiente virtual
- instalar dependências
- testar Ollama localmente

Semana / Etapa 2

- criar FastAPI
- criar endpoint `/health`
- criar `llm_service.py`
- testar chamada ao modelo

Semana / Etapa 3

- criar banco SQLite
- criar tabelas de conversa e memória
- salvar histórico

Semana / Etapa 4

- criar `assistant_service.py`
- montar prompt com contexto
- criar endpoint `/chat`

Semana / Etapa 5

- criar interface com Streamlit ou Gradio
- conectar interface ao backend

Semana / Etapa 6

- criar notas, memórias e tarefas simples
- melhorar logs
- testar fluxo completo

9. Resumo Executivo

O que você vai construir

Uma IA pessoal local, parecida com um Jarvis, com:

- chat
- memória
- contexto
- banco de dados
- interface web
- possibilidade de crescer para ferramentas e automações

Tecnologias certas para começar

- Python
- FastAPI
- Ollama
- SQLite
- Gradio ou Streamlit

Melhor estratégia

Começar simples, modular e funcional.

Primeiro fazer a IA conversar bem.

Depois adicionar memória, documentos, tarefas, ferramentas e automações.

---

10. Próximos Passos Práticos

Faça nessa ordem:

- instalar Python e dependências
- instalar Ollama

- baixar um modelo
- criar projeto FastAPI
- testar endpoint `/health`
- criar serviço do modelo
- criar endpoint `/chat`
- salvar histórico no SQLite
- criar interface web
- adicionar memória

---

11. Bloco pronto para checklist no Notion

Você pode colar esta parte como checklist:

Checklist do Projeto

- [ ] Definir escopo do MVP
- [ ] Criar estrutura de pastas
- [ ] Configurar ambiente virtual
- [ ] Instalar FastAPI
- [ ] Instalar Gradio ou Streamlit
- [ ] Instalar e testar Ollama
- [ ] Baixar modelo local
- [ ] Criar endpoint `/health`
- [ ] Criar serviço `llm_service`
- [ ] Criar banco SQLite
- [ ] Criar tabelas de conversa
- [ ] Criar tabelas de memória
- [ ] Criar `assistant_service`
- [ ] Criar endpoint `/chat`
- [ ] Salvar histórico de mensagens

- [ ] Montar prompt com contexto
- [ ] Criar interface web
- [ ] Conectar interface ao backend
- [ ] Criar sistema de logs
- [ ] Criar ferramentas básicas
- [ ] Adicionar tarefas
- [ ] Preparar futura migração para PostgreSQL

---

12. Estrutura resumida para Notion

Projeto

IA Pessoal Tipo Jarvis

Stack

- Python
- FastAPI
- Gradio ou Streamlit
- Ollama ou llama.cpp
- SQLite
- PostgreSQL futuramente

MVP

- Chat local
- Histórico
- Memória simples
- Interface web
- Banco local

Próximas evoluções

- Tarefas
- Notas
- Arquivos
- RAG
- Voz
- Automação
- PostgreSQL